

0.1 IDPF with half tree optimization

Boneh et al. [1] create incremental distributed functions (IDPF), a new cryptographic tool that builds on standard distributed point functions (DPF), as defined in [2]. IDPF is used to perform prefix count more efficiently, when compared to using DPF for the same application. It is derived from the same algorithm as DPF, modified to provide a per-prefix-level evaluation result while not compromising the security and correctness that the original DPF scheme provides. The construction of IDPF (like DPF) is based on length doubling pseudorandom generators (PRG) to compute the pseudo-random node values of the GGM decision tree [4] used in both schemes.

A more recent development in the field of function secret sharing (FSS) and specifically in DPFs, is the Half-Tree DPF [5]. This improvement manages to halve the cost of GGM decision-tree expansion in DPFs. The improvement is achieved by replacing the GGM tree, which uses pseudo-random nodes, with a correlated GGM (cGGM) tree instead, for which the sum of all same-level nodes equals to a global offset Δ . This invariant is ensured by using a hash function H such that the left child of the parent node x equals $H(x)$ and the right child equals $x - H(x)$. When cGGM trees are used in half-tree DPF, they provide two benefits: (1) the hash function is used only once, halving its computation cost and (2) the length-doubling PRG from GGM tree is exchanged with the aforementioned way of computing each node value, which provides output with the same length as the input, thus halving the computation cost required for each node of the decision tree.

We observe that, the improvements of the DPF scheme using the half-tree technique can be directly used in IDPF to provide the same benefits. In figure 1 we present such a scheme, where the length-doubling PRG of the original IDPF scheme is exchanged with the Half-Tree technique for producing a cGGM tree (htIDPF.Gen algorithm, line 5). This provides a general improvement for IDPF which can impact any application in which IDPF is relevant and manages to halve the cost of computing the nodes of the decision tree. Furthermore, security is not compromised since the same invariants as in DPF hold and the hash function used is a keyed hash function, with the parties holding the key. We hereafter call this improved IDPF version htIDPF.

0.2 Half-Tree IDPF with public β values

In [3] we further observe that IDPF is used for the secure aggregation with the only difference from the original IDPF in [1] being that the IDPF payload (β values) are publicly known. This can lead to a compromise in the security of the scheme. In line 13 of the key generation algorithm in figure 1, the values W_l^{CW} and β_l are publicly known. Also, the values $\langle W_l \rangle_0$ and $\langle W_l \rangle_1$ are generated from a PRG and are members of a group $\mathbb{G}$. Knowing W_l^{CW} and β_l as well as being able to try all the different values for $\langle W_l \rangle_0$ and $\langle W_l \rangle_1$ constrained by the size of the group $\mathbb{G}$, an adversary can reveal the value of the secret $\langle t_l \rangle_1$, which can then be used to determine if the evaluation of any x is on the accepting path of the IDPF decision tree or not. Using this knowledge, the adversary can learn x such that $x = \alpha$, where α is the special evaluation point in DPF such that $f(\alpha) = \beta$.

Based on this observation, we come up with a new htIDPF scheme with public β values, so that the security of the htIDPF scheme is not compromised. This is given in figure 2. In order to hide the secret control bit $\langle t_l \rangle_1$, we introduce a random term $r_l \in \mathbb{G}_l$ per prefix level (line 13, figure 2, key generation algorithm), which is then used to produce shares of $\langle r_l \rangle$ and $\langle \beta_l - r_l \rangle$, which the parties can finally use to securely reveal each level evaluation output β_l .

To do this, we need to introduce the following computation overhead for the key generation and EvalNext algorithms: In the key generation algorithm, we first extend the output length of the function $\text{Convert}_{\mathbb{G}_l}$ to produce 1 extra group $\mathbb{G}_l$ element per prefix level. Secondly, we add an extra group operation so that W_l^{CW} is split into two parts, $\langle W_l^{CW} \rangle_0$ and $\langle W_l^{CW} \rangle_1 \in \mathbb{G}_l$ (representing the shares of $\langle r_l \rangle$ and $\langle \beta_l - r_l \rangle$). Additional to these, a random element needs to be sampled per level of length $|\mathbb{G}_l|$. For the EvalNext algorithm, we likewise introduce one more group operation so that each party can compute y_l^0 and $y_l^1 \in \mathbb{G}_l$, the secret shares $\langle r_l \rangle$ and $\langle \beta_l - r_l \rangle$. Finally, the parties can compute $\langle y_l^0 \rangle_0 + \langle y_l^1 \rangle_0 + \langle y_l^0 \rangle_1 + \langle y_l^1 \rangle_1$ to reconstruct β_l . In total, 2 additional group elements $\mathbb{G}_l$ and an additional group operation per prefix level of the key generation algorithm are needed and an additional group operation is needed in the EvalNext algorithm.

In addition to this increased computational overhead, extra communication overhead is also needed. This is constituted of an extra group element that is attached to each correction word per prefix level (line 16, figure 2, key generation algorithm). Furthermore, the parties need to exchange one extra group element in order to reconstruct the output from the EvalNext algorithm. In total, n group elements from groups $\mathbb{G}_1$ to $\mathbb{G}_n$ plus one group element $\mathbb{G}_l$ from the output level l are exchanged in addition to the communication overhead of the original htIDPF design.

Parameters: Domain size $N = 2^n$ for $n \in \mathbb{N}$. Group $\mathbb{G}$. Keyed hash function $H_S : \mathbb{F}_{2^\lambda} \rightarrow \mathbb{F}_{2^\lambda}$. Function $\text{Convert}_{\mathbb{G}} : \{0, 1\}^* \rightarrow \mathbb{G}$

$\text{htIDPF.Gen}(1^\lambda, (\alpha, (\mathbb{G}_1, \beta_1), \dots, (\mathbb{G}_n, \beta_n)))$

- 1: Let $\alpha = \alpha_1 \dots \alpha_n$, $n \in \{0, 1\}^n$ the bit decomposition of α
- 2: Sample $\Delta \leftarrow \{0, 1\}^\lambda$ such that $\text{lsb}(\Delta) = 1$
- 3: Sample $\langle s_0 \rangle_0, \langle s_0 \rangle_1 \leftarrow \{0, 1\}^\lambda$ & $\langle t_0 \rangle_0 \leftarrow 0, \langle t_0 \rangle_1 \leftarrow 1$ such that $\langle s_0 || t_0 \rangle_0 || \langle s_0 || t_0 \rangle_1 = \Delta$
- 4: **for** $l = 0$ to n **do**:
- 5: $\langle s_l^L || t_l^L \rangle_b || \langle s_l^R || t_l^R \rangle_b = H_S(\langle s_{l-1} || t_{l-1} \rangle_b) || \langle s_{l-1} || t_{l-1} \rangle_b \oplus H_S(\langle s_{l-1} || t_{l-1} \rangle_b), b = 0, 1$
- 6: **if** $\alpha_l = 0$ **then** $\text{keep} \leftarrow L, \text{lose} \leftarrow R$
- 7: **else** $\text{keep} \leftarrow R, \text{lose} \leftarrow L$
- 8: $s_{CW}^{\text{lose}} \leftarrow s_0^{\text{lose}} \oplus s_1^{\text{lose}}$
- 9: $t_{CW}^L \leftarrow t_0^L \oplus t_1^L \oplus \alpha_l \oplus 1$ & $t_{CW}^R \leftarrow t_0^R \oplus t_1^R \oplus \alpha_l$
- 10: $\langle t_l \rangle_b \leftarrow \langle t_l^{\text{keep}} \rangle_b \oplus \langle t_{l-1} \rangle_b \cdot t_{CW}^{\text{keep}}, b = 0, 1$
- 11: $\langle \tilde{s}_l \rangle_b \leftarrow \langle s_l^{\text{keep}} \rangle_b \oplus \langle t_{l-1} \rangle_b \cdot s_{CW}, b = 0, 1$
- 12: $\langle s_l \rangle_b || \langle W_l \rangle_b \leftarrow \text{Convert}_{\mathbb{G}_l}(\langle \tilde{s}_l \rangle_b), b = 0, 1$
- 13: $W_l^{CW} \leftarrow (-1)^{\langle t_l \rangle_1} \cdot [\beta_l - \langle W_l \rangle_0 + \langle W_l \rangle_1]$
- 14: $CW_l \leftarrow s_{CW} || t_{CW}^L || t_{CW}^R || W_l^{CW}$
- 15: Let $k_b \leftarrow \langle s_0 \rangle_b, b = 0, 1$
- 16: Let $\text{pp} \leftarrow CW_1, \dots, CW_n$
- 17: **return** (k_0, k_1, pp)

$\text{htIDPF.EvalPrefix}(b, k_b, \text{pp}, (x_1, \dots, x_l))$

- 1: Let $\langle s_0 \rangle_b \leftarrow k_b$ & $\langle t_0 \rangle_b \leftarrow b$
- 2: $\text{pp} \leftarrow CW_1, \dots, CW_n$
- 3: $\langle st_0 \rangle_b \leftarrow \langle s_0 || t_0 \rangle_b$
- 4: **for** $j = 1$ to l **do**:
- 5: $(\langle st_j \rangle_b, \langle y_j \rangle_b) \leftarrow \text{htIDPF.EvalNext}(b, \langle st_{j-1} \rangle_b, CW_j, x_j)$
- 6: **return** $\langle y_l \rangle_b$

$\text{htIDPF.EvalNext}(b, \langle st_{l-1} \rangle_b, CW_l, x_l)$

- 1: Parse $\langle st_{l-1} \rangle_b = s_{l-1} || t_{l-1}$
- 2: Parse $CW_l = s_{CW} || t_{CW}^L || t_{CW}^R || W_l^{CW}$
- 3: $\hat{s}_l^L || \hat{t}_l^L || \hat{s}_l^R || \hat{t}_l^R \leftarrow H_S(s_{l-1} || t_{l-1}) || s_{l-1} || t_{l-1} \oplus H_S(s_{l-1} || t_{l-1})$
- 4: $\tau_l \leftarrow \hat{s}_l^L || \hat{t}_l^L || \hat{s}_l^R || \hat{t}_l^R \oplus (t_{l-1} \cdot [s_{CW} || t_{CW}^L || s_{CW} || t_{CW}^R])$
- 5: Parse $\tau_l = s_l^L || t_l^L || s_l^R || t_l^R \in \{0, 1\}^{2\lambda+2}$
- 6: **if** $x_l = 0$ **then**: $\tilde{s}_l \leftarrow s_l^L$ & $t_l \leftarrow t_l^L$
- 7: **else**: $\tilde{s}_l \leftarrow s_l^R$ & $t_l \leftarrow t_l^R$
- 8: $s_l || W_l \leftarrow \text{Convert}_{\mathbb{G}_l}(\tilde{s}_l)$
- 9: $st_l \leftarrow s_l || t_l$
- 10: $y_l \leftarrow (-1)^b \cdot [W_l + t_l \cdot W_l^{CW}]$
- 11: **return** st_l, y_l

Figure 1: Adaption of I-DPF based on Half-Tree efficiency improvements

REFERENCES

- [1] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. 2021. Lightweight Techniques for Private Heavy Hitters. In *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 762–776. <https://doi.org/10.1109/SP40001.2021.00048>
- [2] Elette Boyle, Niv Gilboa, and Yuval Ishai. 2016. Function Secret Sharing: Improvements and Extensions. In *ACM CCS 2016*, Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi (Eds.). ACM Press, 1292–1303. <https://doi.org/10.1145/2976749.2978429>

Parameters: Domain size $N = 2^n$ for $n \in \mathbb{N}$. Group $\mathbb{G}$. Keyed hash function $H_S : \mathbb{F}_{2^\lambda} \rightarrow \mathbb{F}_{2^\lambda}$. Function $\text{Convert}_{\mathbb{G}} : \{0, 1\}^* \rightarrow \mathbb{G}$

$\text{pb-htIDPF.Gen}(1^\lambda, (\alpha, (\mathbb{G}_1, \beta_1), \dots, (\mathbb{G}_n, \beta_n)))$

- 1: Let $\alpha = \alpha_1 \dots \alpha_n$, $n \in \{0, 1\}^n$ the bit decomposition of α
- 2: Sample $\Delta \leftarrow \{0, 1\}^\lambda$ such that $\text{lsb}(\Delta) = 1$
- 3: Sample $\langle s_0 \rangle_0, \langle s_0 \rangle_1 \leftarrow \{0, 1\}^\lambda$ & $\langle t_0 \rangle_0 \leftarrow 0, \langle t_0 \rangle_1 \leftarrow 1$ such that $\langle s_0 || t_0 \rangle_0 || \langle s_0 || t_0 \rangle_1 = \Delta$
- 4: **for** $l = 0$ to n **do**:
- 5: $\langle s_l^L || t_l^L \rangle_b || \langle s_l^R || t_l^R \rangle_b = H_S(\langle s_{l-1} || t_{l-1} \rangle_b) || \langle s_{l-1} || t_{l-1} \rangle_b \oplus H_S(\langle s_{l-1} || t_{l-1} \rangle_b), b = 0, 1$
- 6: **if** $\alpha_l = 0$ **then** $\text{keep} \leftarrow L, \text{lose} \leftarrow R$
- 7: **else** $\text{keep} \leftarrow R, \text{lose} \leftarrow L$
- 8: $s_{CW} \leftarrow s_0^{\text{lose}} \oplus s_1^{\text{lose}}$
- 9: $t_{CW}^L \leftarrow t_0^L \oplus t_1^L \oplus \alpha_l \oplus 1$ & $t_{CW}^R \leftarrow t_0^R \oplus t_1^R \oplus \alpha_l$
- 10: $\langle t_l \rangle_b \leftarrow \langle t_l^{\text{keep}} \rangle_b \oplus \langle t_{l-1} \rangle_b \cdot t_{CW}^{\text{keep}}, b = 0, 1$
- 11: $\langle \tilde{s}_l \rangle_b \leftarrow \langle s_l^{\text{keep}} \rangle_b \oplus \langle t_{l-1} \rangle_b \cdot s_{CW}, b = 0, 1$
- 12: $\langle s_l \rangle_b || \langle W_l^0 \rangle_b || \langle W_l^1 \rangle_b \leftarrow \text{Convert}_{\mathbb{G}_l}(\langle \tilde{s}_l \rangle_b), b = 0, 1$
- 13: Sample $r_l \leftarrow \{0, 1\}^{|\mathbb{G}_l|}$
- 14: $\langle W_l^{CW} \rangle_0 \leftarrow (-1)^{\langle t_l \rangle_1} [r - \langle W_l^0 \rangle_0 + \langle W_l^1 \rangle_1]$
- 15: $\langle W_l^{CW} \rangle_1 \leftarrow (-1)^{\langle t_l \rangle_1} [\beta_l - r - \langle W_l^1 \rangle_0 + \langle W_l^1 \rangle_1]$
- 16: $CW_l \leftarrow s_{CW} || t_{CW}^L || t_{CW}^R || \langle W_l^{CW} \rangle_0 || \langle W_l^{CW} \rangle_1$
- 17: Let $k_b \leftarrow \langle s_0 \rangle_b, b = 0, 1$
- 18: Let $\text{pp} \leftarrow CW_1, \dots, CW_n$
- 19: **return** (k_0, k_1, pp)

$\text{pb-htIDPF.EvalPrefix}(b, k_b, \text{pp}, (x_1, \dots, x_l))$

- 1: Let $\langle s_0 \rangle_b \leftarrow k_b$ & $\langle t_0 \rangle_b \leftarrow b$
- 2: $\text{pp} \leftarrow CW_1, \dots, CW_n$
- 3: $\langle st_0 \rangle_b \leftarrow \langle s_0 || t_0 \rangle_b$
- 4: **for** $j = 1$ to l **do**:
- 5: $(\langle st_j \rangle_b, \langle y_j \rangle_b \leftarrow \text{htIDPF.EvalNext}(b, \langle st_{j-1} \rangle_b, CW_j, x_j))$
- 6: **return** $\langle y_l \rangle_b$

$\text{pb-htIDPF.EvalNext}(b, \langle st_{l-1} \rangle_b, CW_l, x_l)$

- 1: Parse $\langle st_{l-1} \rangle_b = s_{l-1} || t_{l-1}$
- 2: Parse $CW_l = s_{CW} || t_{CW}^L || t_{CW}^R || \langle W_l^{CW} \rangle_0 || \langle W_l^{CW} \rangle_1$
- 3: $\hat{s}_l^L || \hat{t}_l^L || \hat{s}_l^R || \hat{t}_l^R \leftarrow H_S(s_{l-1} || t_{l-1}) || s_{l-1} || t_{l-1} \oplus H_S(s_{l-1} || t_{l-1})$
- 4: $\tau_l \leftarrow \hat{s}_l^L || \hat{t}_l^L || \hat{s}_l^R || \hat{t}_l^R \oplus (t_{l-1} \cdot [s_{CW} || t_{CW}^L || s_{CW} || t_{CW}^R])$
- 5: Parse $\tau_l = s_l^L || t_l^L || s_l^R || t_l^R \in \{0, 1\}^{2\lambda+2}$
- 6: **if** $x_l = 0$ **then**: $\tilde{s}_l \leftarrow s_l^L$ & $t_l \leftarrow t_l^L$
- 7: **else**: $\tilde{s}_l \leftarrow s_l^R$ & $t_l \leftarrow t_l^R$
- 8: $s_l || W_l^0 || W_l^1 \leftarrow \text{Convert}_{\mathbb{G}_l}(\tilde{s}_l)$
- 9: $st_l \leftarrow s_l || t_l$
- 10: $y_l^0 \leftarrow (-1)^b \cdot [W_l^0 + t_l \cdot \langle W_l^{CW} \rangle_0]$
- 11: $y_l^1 \leftarrow (-1)^b \cdot [W_l^1 + t_l \cdot \langle W_l^{CW} \rangle_1]$
- 12: **return** st_l, y_l^0, y_l^1

Figure 2: Half-Tree I-DPF with public beta values

- [3] Nan Cheng, Aikaterini Mitrokotsa, Feng Zhang, and Frank Hartmann. 2024. Efficient Two-Party Secure Aggregation via Incremental Distributed Point Function. In *2024 IEEE European Symposium on Security and Privacy*. IEEE Computer Society Press, 810–827. <https://doi.org/10.1109/EuroSP60621.2024.00050>
- [4] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. 1986. How to Construct Random Functions. *Journal of the ACM* 33, 4 (Oct. 1986), 792–807. <https://doi.org/10.1145/6490.6503>
- [5] Xiaojie Guo, Kang Yang, Xiao Wang, Wenhao Zhang, Xiang Xie, Jiang Zhang, and Zheli Liu. 2023. Half-Tree: Halving the Cost of Tree Expansion in COT and DPF. In *EUROCRYPT 2023, Part I (LNCS, Vol. 14004)*, Carmit Hazay and Martijn Stam (Eds.). Springer, Cham, 330–362. https://doi.org/10.1007/978-3-031-30545-0_12